P1682R1
std::to_underlying for enumerations

Published Proposal, 2019-08-05

Author:
JeanHeyd Meneide

Audience:
LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:
C++23

Latest:
https://thephd.github.io/vendor/future_cxx/papers/d1682.html

Abstract

A proposal to add a short utility function to handle going from an enumeration to its underlying integral value for safety and ease of use.

Table of Contents

1 Revision History

1.1 Revision 1 - August 5th, 2019

1.2 Revision 0 - June 17th, 2019

2 Motivation

3 Design

4 Proposing Wording

4.1 Proposed Feature Test Macro
4.2 Intent
4.3 Proposed Library Wording

5 Acknowledgements

References
Informative References

§ 1. Revision History

§ 1.1. Revision 1 - August 5th, 2019

— Approved for Library Working Group in the Köln, Germany 2019 meeting!

§ 1.2. Revision 0 - June 17th, 2019

— Initial release.

§ 2. Motivation

Many codebases write a version of a small utility function converting an enumeration to its underlying type. The reason for this function is very simple: applying `static_cast<int>`/ `static_cast<unsigned long>` (or similar) to change an enumeration to its underlying type makes it harder to quickly read and maintain places where the user explicitly converts from a strongly-typed enumeration to its underlying value. For the purposes of working with an untyped API or similar, casts just look like any old cast, making it harder to read code and potentially incorrect when enumeration types are changed from signed / unsigned or similar.

Much of the same rationale is why this is Item 10 in Scott Meyers' Effective Modern C++. In Around Christmas of 2016, the number of these function invocations for C++ was around 200 including both to_underlying/to_underlying_type/toUtype (the last in that list being the way it was spelled by Scott Meyers). As of June 17th, 2019, the collective hits on GitHub and other source engines totals well over 1,000 hits, disregarding duplication from common base frameworks such as the realm mobile app database and more. The usefulness of this function appears in Loggers for enumerations, casting for C APIs, stream operations, and more.

We are seeing an explosive move and growth in usage of Modern C++ utilities, and the growth in this usage clearly indicates that the foresight and advice of Scott Meyers is being taken seriously by the full gamut of hobbyist to commercial software engineers. Therefore, it would seem prudent to make the spelling and semantics of this oft-reached-for utility standard in C++.

Typical casts can also mask potential bugs from size/signed-ness changes and hide programmer intent. For example, going from this code,

```
enum class ABCD {
        A = 0x1012,
        B = 0x405324,
        C = A & B
};

// sometime later ...

void do_work(ABCD some_value) {
        // no warning, no visual indication,
        // is this what the person wanted,
        // what was the original intent in this
        // 'harmless' code?
        internal_untyped_api(static_cast<int>(some_value));
}
```

To this code:

```
#include <cstdint>

// changed enumeration, underlying type
enum class ABCD : uint32_t {
        A = 0x1012,
        B = 0x405324,
        C = A & B,
        D = 0xFFFFFFFF // !!
};

// from before:

void do_work(ABCD some_value) {
        // no warning, no visual indication,
        // is this what the person wanted,
        // what was the original intent in this
        // 'harmless' code?
        internal_untyped_api(static_cast<int>(some_value));
}
```

is dangerous, but the `static_cast<int>` is seen by the compiler as intentional by the user.

Calling `do_work(ABCD::D);` is a code smell internally because the cast is the wrong one for the enumeration. If the internal untyped API takes an integral value larger than the size of `int` and friends, then this code might very well pass a bit pattern that will be interpreted as the wrong value inside of the `internal_untyped_api`, too. Of course, this change does not trigger warnings or errors: `static_cast<int>` is a declaration of intent that says "I meant to do this cast", even if that cast was done before any changes or refactoring was performed on the enumeration.

Doing it the right way is also cumbersome:

```
void do_work(ABCD some_value) {
        // will produce proper warnings,
        // but is cumbersome to type
        internal_untyped_api(static_cast<std::underlying_type_t<ABCD>>(some
}
```

It is also vulnerable to the parameter's type changing from an enumeration to another type that is convertible to an integer. Because it is still a `static_cast`, unless someone changes the type for `do_work` while also deleting `ABCD`, that code will still compile:

```
void do_work(OtherEnumeration value) {
        // no warnings, no errors, ouch!
        internal_untyped_api(static_cast<std::underlying_type_t<ABCD>>(some
}
```

We propose an intent-preserving function used in many codebases across C++ called `std::to_underlying`, to be used with enumeration values.

§ 3. Design

`std::to_underlying` completely avoids all of the above-mentioned problems related to code reuse and refactoring. It makes it harder to write bugs when working with strongly-typed enumerations into untyped APIs such with things such as C code and similar. It only works on enumeration types. It will `static_cast` the enumeration to integral representation with `std::underlying_type_t<T>`. This means that the value passed into the function provides the type information, and the type information is provided by the compiler, not by the user.

This makes it easy to find conversion points for "unsafe" actions, reducing search and refactoring area. It also puts the `static_cast` inside of a utility function, meaning that warnings relating to size and signed-ness differences can still be caught in many cases since the result's usage comes from a function, not from an explicitly inserted user cast.

```
#include <utility>

void do_work(MyEnum value) {
        // changes to match its value,
        // proper warnings for signed/unsigned mismatch,
        // and ease-of-use!
        internal_untyped_api(std::to_underlying(some_value));
}
```

§ 4. Proposing Wording

The wording proposed here is relative to [n4800].

§ 4.1. Proposed Feature Test Macro

The proposed library feature test macro is `__cpp_lib_to_underlying`.

§ 4.2. Intent

The intent of this wording is to introduce 1 function into the `<utility>` header called `to_underlying`. If the input to the function is not an enumeration, then the program is ill-formed.

§ 4.3. Proposed Library Wording

Append to §17.3.1 General [**support.limits.general**]'s **Table 35** one additional entry:

Macro name	Value
__cpp_lib_to_underlying	202002L

Add the following into §20.2.1 Header `<utility>` [utility.syn] synopsis:

```
// [utility.underlying], to_underlying
template <class T>
    constexpr std::underlying_type_t<T> to_underlying( T value ) noexcep
```

Add a new section §20.2.7 Function template `to_underlying` [utility.underlying]:

20.2.7 Function template `to_underlying` [**utility.underlying**]

```
namespace std {
        template <typename T>
        constexpr std::underlying_type_t<T> to_underlying( T value ) noe
}
```

[1] Constraints: `T` shall satisfy `std::is_enum_v<T>`.

[2] Returns: `static_cast<std::underlying_type_t<T>>(value)`.

§ 5. Acknowledgements

Thanks to Rein Halbersma for bringing this up as part of the things that would make programming in his field easier and the others who chimed in. Thanks to Walter E. Brown for the encouragement to Rein Halbersma to get this paper moving.

§ References

§ Informative References

[N4800]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4800 - Working Draft, Standard for Programming Language C++. January 21st, 2019. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4800.pdf